Deep Q-Networks (DQN) by Hand: A Fully Worked, Classroom Example

Instructor: Ahmed Métwalli

A compact, classroom-friendly introduction to Deep Q-Networks (DQN) is provided. The conceptual move from tabular Q-learning to function approximation is highlighted. The target network and replay buffer are defined as stabilizing components. A fully numeric, hand-computable example is then presented using assumed network outputs so that the DQN target, temporal-difference (TD) error, and squared loss can be computed explicitly. A terminal-state variant and a small two-sample mini-batch extension are also included. A practical "when to use what" comparison with tabular Q-learning is provided.

DQN at a Glance: Motivation and Core Idea

Motivation. When the state–action space is too large for a table, the action-value function is approximated with a neural network.

Core replacement. The tabular entry $Q(s, a)$ is replaced by a parametric function:

$$Q(s, a; \theta),$$

where θ denotes the learnable parameters of a neural network (the *online* network).

Model-free. A transition model $p(s'|s, a)$ is not required; learning is driven by sampled experience.

Stability components.

- A *replay buffer* is used to store transitions (s, a, r, s') and to sample mini-batches for training.
- A *target network* with parameters θ^- is used to compute more stable targets.
- Target parameters are periodically copied: $\theta^- \leftarrow \theta$.

Action-space note. Classic DQN is intended for *discrete* actions.

DQN Update Rule (Single-Sample Form)

For a transition (s, a, r, s'), the target is defined as

$$y = \begin{cases} r, & \text{if } s' \text{ is terminal,} \\ r + \gamma \max_{a'} Q(s', a'; \theta^-), & \text{otherwise.} \end{cases}$$

The squared error loss for one sample is written as

$$L(\theta) = \big(y - Q(s, a; \theta)\big)^2.$$

A gradient step is applied to reduce $L(\theta)$ using suitable optimizers.

A Tiny Environment Context (for Notation Only)

A 3×3 gridworld setting may be assumed for intuition. States may be denoted by coordinates (i, j), with actions *up, down, left, right*. A terminal goal state may be placed at $(2, 2)$. This section is included only to supply interpretable state labels; the DQN arithmetic below is network-based and does not require a full grid rollout.

Hand-Computable DQN Example (Non-Terminal)

Given.

- Discount factor: $\gamma = 0.9$
- A single transition:
$$s = (1, 1), \quad a = \text{Right}, \quad r = 1, \quad s' = (1, 2),$$
- The next state is assumed *non-terminal*.

Assumed network outputs. These values are treated as *given numbers* produced by the networks:

$$Q(s, a; \theta) = Q((1, 1), \text{Right}; \theta) = 0.5,$$

$$Q((1, 2), \text{Up}; \theta^-) = 0.8, \quad Q((1, 2), \text{Down}; \theta^-) = 0.3, \quad Q((1, 2), \text{Left}; \theta^-) = 0.1, \quad Q((1, 2), \text{Right}; \theta^-) = 0.2.$$

Hence,

$$\max_{a'} Q(s', a'; \theta^-) = 0.8.$$

Target calculation.

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-) = 1 + 0.9 \times 0.8 = 1 + 0.72 = 1.72.$$

TD error.

$$\delta = y - Q(s, a; \theta) = 1.72 - 0.5 = 1.22.$$

Squared loss (single sample).

$$L(\theta) = \delta^2 = 1.22^2 = 1.4884.$$

Interpretation. An underestimation of the value of taking *Right* in $(1, 1)$ is indicated. A gradient update is therefore applied so that $Q((1, 1), \text{Right}; \theta)$ is moved upward toward 1.72.

Hand-Computable DQN Example (Terminal Variant)

Given.

$$s = (1, 2), \quad a = \text{Down}, \quad r = 10, \quad s' = (2, 2) \text{ (terminal)}.$$

Let

$$Q(s, a; \theta) = Q((1, 2), \text{Down}; \theta) = 5.0.$$

Target. Since s' is terminal,

$$y = r = 10.$$

TD error and loss.

$$\delta = 10 - 5.0 = 5.0, \quad L(\theta) = 5.0^2 = 25.$$

Mini-Batch Extension (Two Samples, Fully Computed)

A small mini-batch may be used to demonstrate how DQN losses are aggregated.

Sample 1 (non-terminal). This sample is reused from the previous non-terminal example:

$$y_1 = 1.72, \quad Q_1 = 0.5, \quad \delta_1 = 1.22, \quad L_1 = 1.4884.$$

Sample 2 (terminal). This sample is reused from the terminal example:

$$y_2 = 10, \quad Q_2 = 5.0, \quad \delta_2 = 5.0, \quad L_2 = 25.$$

Mean squared loss over the mini-batch.

$$\bar{L} = \frac{L_1 + L_2}{2} = \frac{1.4884 + 25}{2} = \frac{26.4884}{2} = 13.2442.$$

Tabular Q-Learning vs. DQN (What Is Changed and When Each Is Preferred?)

Aspect	**Q-Learning**	**DQN**
Representation	Table $Q(s, a)$	Network $Q(s, a; \theta)$
Target	$r + \gamma \max_{a'} Q(s', a')$	$r + \gamma \max_{a'} Q(s', a'; \theta^-)$
Update mechanism	Direct value update	Loss minimization via gradients
Stability tools	Often not needed in small tabular MDPs	Replay buffer + target network are typically required
Action space	Discrete actions	Discrete actions (classic DQN setting)
Primary use case	Small, finite state–action spaces where a table fits	Large/high-dimensional state spaces where a table is infeasible
Compute needs	Low; simple hardware is often sufficient	Moderate to high; GPU acceleration is commonly beneficial
Interpretability	High; each entry can be inspected	Lower; values are embedded in network parameters
When preferred	• Fundamental TD control concepts are being taught. • Toy/grid environments are being used for demonstrations. • Reward shaping and dynamics checks are being debugged transparently. • State encodings remain small and discrete.	• State spaces are too large to be tabulated reliably. • High-dimensional observations (e.g., pixels) are provided. • Generalization across unseen states is required. • Performance beyond tabular scaling limits is needed.

Practical selection rule. A simple rule of thumb may be stated:

- **Tabular Q-learning should be used** when the state space is small enough that a reliable table can be stored and explored, and when interpretability and step-by-step hand calculation are desired.
- **DQN should be used** when the state space is too large for a table, or when high-dimensional inputs are given and useful generalization across unseen states is required.

Important boundary. Classic DQN is intended for *discrete* action spaces. When actions are continuous, actor–critic methods (e.g., DDPG, TD3, SAC) are typically preferred.

Minimal DQN Training Skeleton (Narrative)

A typical DQN loop may be summarized as follows:

- Transitions (s, a, r, s') are collected using an exploratory behavior policy (often ε-greedy).

- Transitions are stored in a replay buffer.
- Mini-batches are sampled from the buffer.
- Targets y are computed using the target network.
- The online network is updated by minimizing $\big(y - Q(s, a; \theta)\big)^2$ across the mini-batch.
- The target network is periodically synchronized with the online network.

What Should Be Remembered

- DQN is a *model-free, value-based, off-policy* method.
- The Bellman target idea is preserved from Q-learning.
- The table is replaced by a function approximator trained to match that target.
- The target network reduces oscillations by providing a slowly moving reference.
- The replay buffer improves data efficiency and reduces harmful correlation.
- A hand-computable DQN example can be constructed by assuming network outputs, then computing y, δ, and L explicitly.